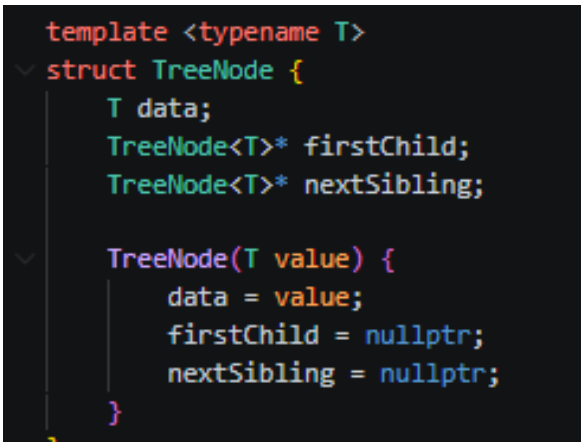
COMPUTER ENGINEERING

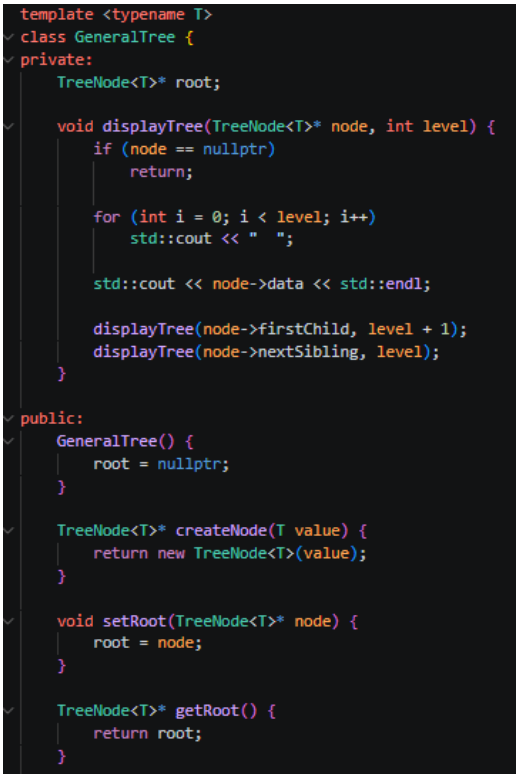
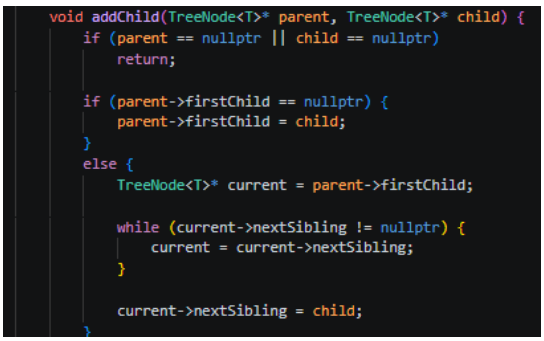
QUEZON CITY

COURSE CODE/TITLE:	CpE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S1
DATE PERFORMED:	9-21-2026	**DATE SUBMITTED:**	9-23-2026
NAME:	Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 **[Trees ADT]**

1. PROCEDURE OUTPUT

Screenshot of Code	Explanation
	This structure represents each node in the general tree. It stores the node's data, a pointer to its first child, and a pointer to its next sibling.

	The GeneralTree class manages the general tree. The root pointer stores the starting node of the tree, while createNode() creates new nodes dynamically.
	The addChild() function connects a child node to its parent. The first child is stored using firstChild, while additional children are connected using nextSibling.
	The displayTree() function recursively displays the nodes of the tree. The level is used to show the hierarchy of the nodes through indentation.

```
int main() {

    GeneralTree<char> tree;

    // Create the root node
    TreeNode<char>* A = tree.createNode('A');
    tree.setRoot(A);

    // Create nodes B to G
    TreeNode<char>* B = tree.createNode('B');
    TreeNode<char>* C = tree.createNode('C');
    TreeNode<char>* D = tree.createNode('D');
    TreeNode<char>* E = tree.createNode('E');
    TreeNode<char>* F = tree.createNode('F');
    TreeNode<char>* G = tree.createNode('G');

    // Create nodes H to N
    TreeNode<char>* H = tree.createNode('H');
    TreeNode<char>* I = tree.createNode('I');
    TreeNode<char>* J = tree.createNode('J');
    TreeNode<char>* K = tree.createNode('K');
    TreeNode<char>* L = tree.createNode('L');
    TreeNode<char>* M = tree.createNode('M');
    TreeNode<char>* N = tree.createNode('N');

    // Create nodes P and Q
    TreeNode<char>* P = tree.createNode('P');
    TreeNode<char>* Q = tree.createNode('Q');
```

This section creates all the nodes needed to reproduce the general tree in the given figure. Node A is assigned as the root of the tree.

```
tree.addChild(A, B);
tree.addChild(A, C);
tree.addChild(A, D);
tree.addChild(A, E);
tree.addChild(A, F);
tree.addChild(A, G);

// Add H as child of D
tree.addChild(D, H);

// Add I and J as chil
tree.addChild(E, I);
tree.addChild(E, J);

// Add P and Q as chil
tree.addChild(J, P);
tree.addChild(J, Q);

// Add K, L, and M as
tree.addChild(F, K);
tree.addChild(F, L);
tree.addChild(F, M);

// Add N as child of G
tree.addChild(G, N);
```

This section creates the same parent-child relationships shown in the given diagram. B, C, D, E, F, and G are children of A, while the remaining nodes are connected to their corresponding parents.

``
// Display the tree
std::cout << "General Tree:" << std::endl;
tree.display();
``
	The display() function is called to show the completed general tree. The program then ends using return 0.

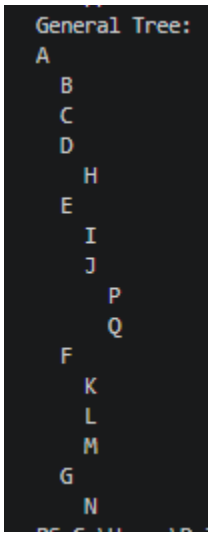
	The indentation represents the levels of the tree.

Table 9-1. ILOA Task 1

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2

L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2. ILO A Task 2

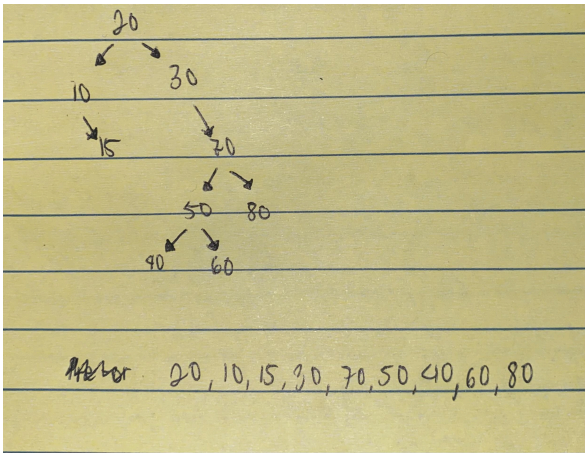
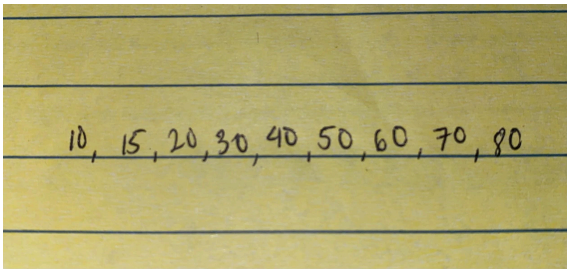
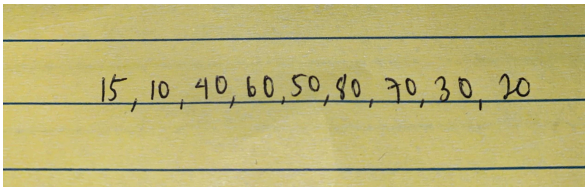
Pre-order	
In-order	
Post-order	

Table 9-3. ILO B Task 3

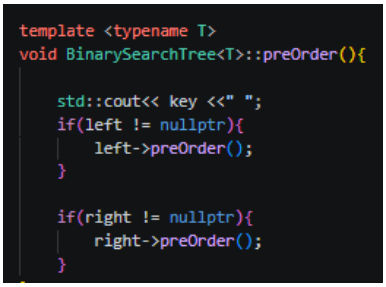
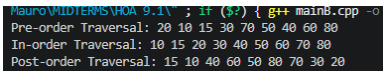
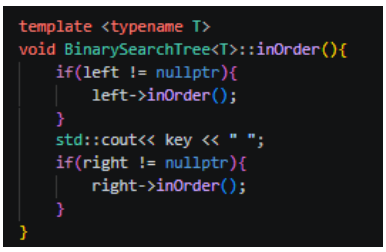
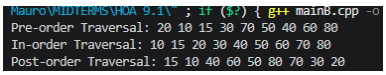
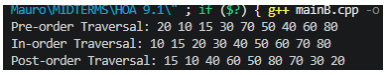
Screenshot of code	Output	Observation
		The function starts at the root, then visits the left subtree before visiting the right subtree. The output is the same as the pre-order result calculated by hand in Table 9-3.
		The function visits the left subtree first, followed by the root, and then the right subtree. The output matches the in-order result calculated by hand.
		The function visits the left and right subtrees before visiting the root. The output matches the post-order result calculated by hand.

Table 9-4. ILO B Task 3

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

For Step 1:

```
int main()
{
    BST<int> tree(2);

    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    std::cout << "In-order Traversal: ";
    tree.inOrder();
    std::cout << std::endl;

    std::cout << "Pre-order Traversal: ";
    tree.preOrder();
    std::cout << std::endl;

    std::cout << "Post-order Traversal: ";
    tree.postOrder();
    std::cout << std::endl;

    return 0;
```

```
#define BST_H

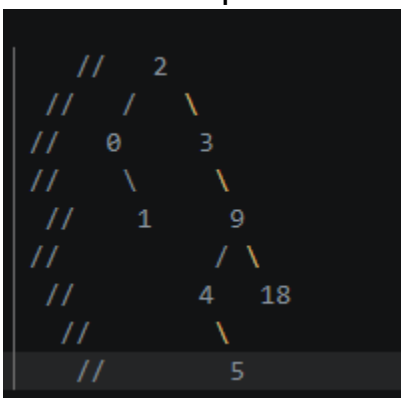
#include <iostream>

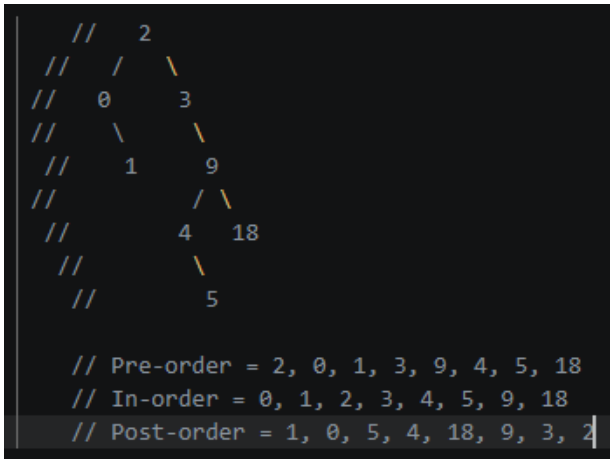
template <typename T>
class BST
{
private:
    T key;
    BST<T>* leftChild;
    BST<T>* rightChild;

public:
    BST(T value)
    {
        key = value;
        leftChild = nullptr;
        rightChild = nullptr;
    }

    void insert(T value)
    {
        if (value < key)
        {
            if (leftChild == nullptr)
            {
                leftChild = new BST<T>(value);
            }
            else
            {
                leftChild->insert(value);
            }
        }
        else if (value > key)
        {
            if (rightChild == nullptr)
            {
                rightChild = new BST<T>(value);
            }
            else
            {
                rightChild->insert(value);
            }
        }
    }
```

For Step 2:





For Step 3:

```
void preOrder()
{
    std::cout << key << " ";

    if (leftChild != nullptr)
    {
        leftChild->preOrder();
    }

    if (rightChild != nullptr)
    {
        rightChild->preOrder();
    }
}
```

```
In-order Traversal: 0 1 2 3 4 5 9 18
Pre-order Traversal: 2 0 1 3 9 4 5 18
```

The output is the same as the pre-order traversal indicated in the diagram. The root node 2 is visited first, followed by the left subtree and then the right subtree.

```
void inOrder()
{
    if (leftChild != nullptr)
    {
        leftChild->inOrder();
    }

    std::cout << key << " ";

    if (rightChild != nullptr)
    {
        rightChild->inOrder();
    }
}
```

```
In-order Traversal: 0 1 2 3 4 5 9 18
```

```
void postOrder()
{
    if (leftChild != nullptr)
    {
        leftChild->postOrder();
    }

    if (rightChild != nullptr)
    {
        rightChild->postOrder();
    }

    std::cout << key << " ";
}
```

```
Post-order Traversal: 1 0 5 4 18 9 3 2
```

The output is the same as the post-order traversal indicated in the diagram. The left and right subtrees are visited before their parent nodes, and the root node 2 is visited last.

Conclusion:

In this activity, I learned how to create and use a Binary Search Tree in C++. By inserting the given values, I was able to form the correct tree structure and perform the three traversal methods: in-order, pre-order, and post-order. I learned that each method visits the nodes in a different order: in-order follows Left -Root -Right, pre-order follows Root -Left -Right, and post-order follows Left - Right -Root. I also learned how recursion can be used to visit each node in the tree. The in-order traversal showed the values in ascending order, while the pre-order and post-order traversals produced different sequences based on how the nodes were visited. Overall, this activity helped me better understand how Binary Search Trees work and how traversal methods can be implemented in C++.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **none**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **none**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **none**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.